Lecture 25: SQL as a Calculator

Formatting, Math, and Basic Statistics

Z2004: Database Management Systems

D0.jpg

IIT Madras Zanzibar

Tuesday, 28 April 2026

Agenda for Today

Transforming the Data You Query!

1. **Pop Quiz:** Remembering `ORDER BY` and `LIKE`.
2. **Dressing Up Data:** Column Aliases (`AS`).
3. **String Functions:** `UPPER()`, `LOWER()`, and `LEN()`.
4. **Basic Math:** Doing calculations directly in your `SELECT`.
5. **The Big 5 Aggregates:** `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

Mini-Quiz: Sorting and Searching!

Question 1: What will this query do?

```
SELECT Name
FROM Student
WHERE Name LIKE '%Hassan%';
```

Question 2: You want to see the 3 students with the **highest** GPAs. What is wrong with this query?

```
SELECT TOP 3 Name, GPA FROM Student
ORDER BY GPA ASC;
```

Question 3: Write a query to list all students whose department starts with the letter `C`, sorted by `Name` A–Z.

Mini-Quiz: Answers!

Answer 1: It returns the name of any student that has the word 'Hassan' **anywhere** in their name (e.g., Maryam Hassan).

Answer 2: `ASC` means ascending (lowest to highest). So this query gives the 3 **lowest** GPAs! To get the highest, you should use `DESC` (Descending):

```sql
ORDER BY GPA DESC;
```

Answer 3: Use `LIKE` with a prefix pattern and then sort:

```sql
SELECT Name, Dept
FROM Student
WHERE Dept LIKE 'C%'
ORDER BY Name ASC;
```

1. Dressing up Data: Aliases (AS)

When we start doing math or using functions, SQL gives the output column an ugly name (like '(No column name)'). We can rename it for the output using AS.

```
SELECT
    Name AS FullName,
    Dept AS Department
FROM Student;
```

FullName	Department
Asha Ali	CSE
Zain Omar	EEE

Note: This does not change the actual table structure in the database. It only changes how the results look on your screen!

2. String Functions

SQL can edit text on the fly before it shows it to you.

```sql
SELECT TOP 3
    UPPER(Name) AS YellingName,
    LOWER(Dept) AS QuietDept,
    LEN(Name) AS NameLength
FROM Student;
```

Output:

YellingName	QuietDept	NameLength
ASHA ALI	cse	8
ZAIN OMAR	eee	9
MARYAM HASSAN	me	13

3. Basic Math in SQL

You can use standard math symbols ($+, -, *, /$) directly inside the SELECT clause!

Scenario: A student's current GPA is out of 4.0. We want to convert it to a percentage (out of 100).

```
SELECT TOP 3
    Name,
    GPA,
    (GPA / 4.0) * 100 AS GPAPercentage
FROM Student;
```

Output:

Name	GPA	GPAPercentage
Asha Ali	3.80	95.000000
Zain Omar	3.45	86.250000
Maryam Hassan	3.12	78.000000

4. The Big 5 Aggregate Functions

Instead of calculating something row-by-row, what if we want a statistic for the **entire table**?

- `COUNT(*)`: How many total rows?
- `SUM(column)`: Add all the numbers together.
- `AVG(column)`: The mathematical average.
- `MIN(column)`: The lowest value.
- `MAX(column)`: The highest value.

```sql
SELECT
    COUNT(*) AS TotalStudents,
    AVG(GPA) AS SchoolAverage,
    MAX(GPA) AS HighestGPA
FROM Student;
```

Notice: This collapses the entire table into a single row of summary statistics!

Aggregate Examples (SchoolDB): COUNT, AVG, MIN, MAX

Student table (Student(SID, Name, Dept, GPA, ...))

```sql
-- How many students are in the database?
SELECT COUNT(*) AS TotalStudents
FROM Student;

-- What is the average GPA (NULL GPAs are ignored)?
SELECT AVG(GPA) AS AvgGPA
FROM Student;

-- Lowest and highest GPA in the table
SELECT MIN(GPA) AS LowestGPA, MAX(GPA) AS HighestGPA
FROM Student;
```

Aggregate Examples (SchoolDB): SUM

Course table (`Course(CID, CName, Credits)`)

```sql
-- Total credits offered across all courses
SELECT SUM(Credits) AS TotalCreditsOffered
FROM Course;

-- Total credits a particular student is enrolled in
SELECT e.SID, SUM(c.Credits) AS TotalCreditsEnrolled
FROM Enrolment e
JOIN Course c ON c.CID = e.CID
WHERE e.SID = 1;
```

Practical Guidelines

- **NULLs and aggregates:**
  - `COUNT(*)` counts rows; `COUNT(GPA)` ignores NULL GPAs.
  - `AVG(GPA)` ignores NULL GPAs.
- **Integer division:** in many DBs, `3/4 = 0`. Use a decimal like `4.0` or `CAST(...AS DECIMAL)`.
- **WHERE vs HAVING:** `WHERE` filters rows *before* grouping; `HAVING` filters groups *after* grouping.

```
-- Example: COUNT(*) vs COUNT(GPA)
SELECT COUNT(*) AS Rows, COUNT(GPA) AS NonNullGPAs
FROM Student;
```

Practice (Write These Queries)

Use our SchoolDB tables: `Student`, `Course`, `Enrolment`

1. List **CSE** students with **GPA** $\geq$ **3.0**, show `Name, GPA`, sort by GPA (highest first).
2. Show each student's GPA percentage: `(GPA/4.0)*100` as `GPAPercentage`; show top 5.
3. How many students are in each department? (Dept-wise count)

Try for 5 minutes, then we'll check the answers.

Practice: Sample Answers

```sql
-- 1) CSE students with GPA >= 3.0, highest GPA first
SELECT Name, GPA
FROM Student
WHERE Dept = 'CSE  ' AND GPA >= 3.0
ORDER BY GPA DESC;

-- 2) GPA percentage, top 5
SELECT TOP 5 Name, GPA, (GPA / 4.0) * 100 AS GPAPercentage
FROM Student
WHERE GPA IS NOT NULL
ORDER BY GPAPercentage DESC;

-- 3) Students per department
SELECT Dept, COUNT(*) AS NumStudents
FROM Student
GROUP BY Dept
ORDER BY NumStudents DESC;
```

Aggregates with GROUP BY and HAVING

```sql
-- Average GPA per department
SELECT Dept, AVG(GPA) AS AvgGPA
FROM Student
GROUP BY Dept;

-- Only show departments with at least 2 students
SELECT Dept, COUNT(*) AS NumStudents
FROM Student
GROUP BY Dept
HAVING COUNT(*) >= 2;
```

Cleaner Output: Rounding and Aliases

```sql
-- Round to 2 decimals for a nicer report
SELECT
    COUNT(*)               AS TotalStudents,
    ROUND(AVG(GPA), 2)     AS AvgGPA,
    MIN(GPA)               AS LowestGPA,
    MAX(GPA)               AS HighestGPA
FROM Student;
```

Summary

Key takeaways (L25: SQL as a Calculator)

- `AS`: Renames a column in your output to make it easier to read.
- **Functions**: SQL has built-in tools like `UPPER()` and `LOWER()` to format strings.
- **Math**: You can do math directly in the `SELECT` line (e.g., `Price * 1.15 AS PriceWithTax`).
- **Aggregates**: You can use `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX` to summarize the entire table into a single row of statistics.